

SECTION A

A1. Identify the issues and list it with reasons

The issues seen in the given schema is

- There are many entities like users, movies, show, theatre etc.. everything is used in the same table
- Some attributes have no direct relation between the table
- We must have different table for different entities.. like storing user_email in booking makes the database design messy. Instead we can create a new table for user id, user name and user email. Similarly we have to create multiple tables for different entities
- There are multiple values for same attribute such that if a person books 3 seats then they will be multi-valued and it violates 1NF form
- There is no id's mentioned for users, movies, theatre etc..
- Since 1NF form is violated, this scheme doesn't follow 2NF or 3NF.
- Everything is being used in a single table resulting in messy table, instead we have to create multiple tables

A2. Normalize the Schema

1NF states that there should not be atomic values or multiple values.. here seat number violates that rule hence a new table is to be created.

2NF states that partial dependency should not be present. Here many different attributes belong to the same table hence different tables are to be constructed such as users, movies, etc..

3NF states that transitive dependencies should not be present but in the given schema this rule is violated

Therefore,

The tables to be created are

- Users
- Movies
- Theatre
- Show
- Screen
- Seat
- Bookings
- Booking_seats
- Payment

These tables include

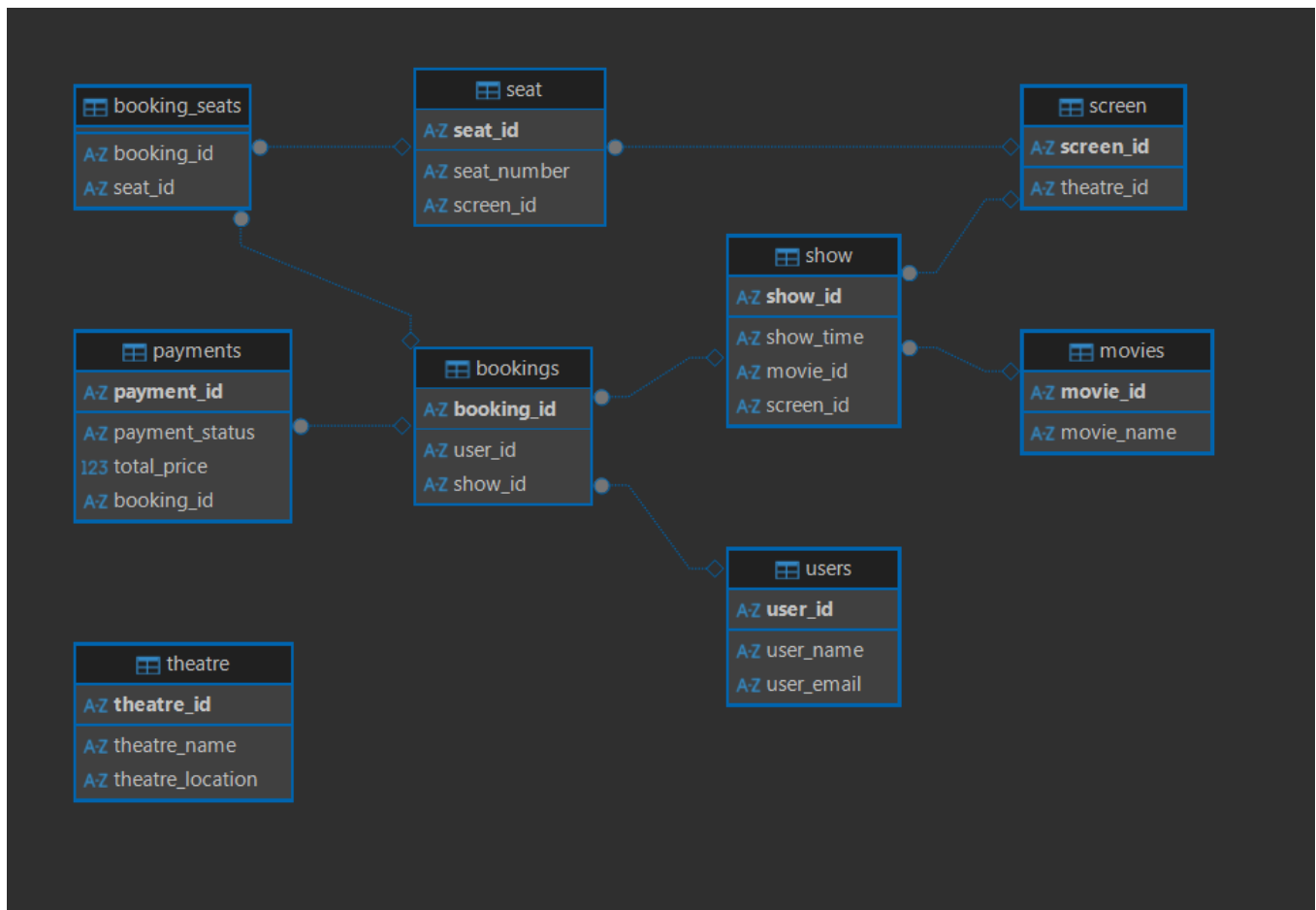
- Users include user_name, user_id [Primary key], and user_email.
- Movies include movie_id [Primary key], movie_name
- Theatre includes theatre_id [Primary key], theatre name, theatre_location
- Show includes show_id [Primary key], show_time, movie_id [foreign key], screen_id [foreign key]
- Screen includes screen_id [Primary key], Theatre_id [foreign key]
- Seat includes seat_id [Primary key], seat_number, screen_id [foreign key]

- Bookings includes booking_id[Primary key], user_id[foreign key], show_id[foreign key]
- Booking_seats includes booking_id[foreign key], seat_id[foreign key]
- Payment includes payment_id[Primary key], payment_status, total_price, booking_id[foreign key]

A3. Assumptions

- There are unique id's in every table that is needed as primary key
- One user can book multiple movies.
- One theater can have multiple screens
- One booking can have many seatings hence a table booking seats are included.
- Total price is calculated for whole booking not just seat

ER Diagram



SECTION B

B1. Requirement Clarification

- How many account can be present per email..?
- Can booking include multiple users or only one..?
- Can the same seat be booked multiple times..?
- Does each theatre belong to the same city.?
- Can same movie have many languages and genres at the same time..?
- Does pricing depend on show timing or seat type too..?
- Can one show have different prices for different seat types..?
- Should we also track booking date and time..?
- Can multiple movies run in multiple theatres at the same time.?
- How is seat availability tracked in the system.?

B2. Design

The design can be given as

- The tables required are
 - Users
 - Movies
 - City
 - Theatre
 - Screen
 - Shows
 - Seat
 - Bookings
 - Booking_seats
 - Payments
 - Genre
 - Language
- These tables include
 - Users – user_id[PK], user_name and user_email
 - Movies – movie_id [PK], movie_name
 - City – city_id [PK], city_name
 - Theater – theatre_id[PK] , theatre_name, city_id[Fk]
 - Screen – screen_id[PK], theatre_id[FK]
 - Shows – show_id[pK], show_time, price, movie_id[FK], screen_id[FK]
 - Seat – seat_id[PK], seat_number, seat_type and screen_id
 - Bookings – booking_id[PK], user_id[FK], show_id[FK]
 - Booking_seats – booking_seat and seat_id (Both are primary keys)
 - Payments – payment_id[PK], payment_status, total_price, booking_id[FK]
 - Genre – movie_id[Fk], genre_id(Both are primary keys)
 - Language – movie_id[FK], language_name(both are primary keys)

B3. Normalization

1NF states that table must not contain the atomic values. Hence seats are stored in different table instead of storing it using commas and also genres and languages are stored in separate rows. Hence 1NF is satisfied.

2NF states that no partial dependency should be present. In booking_seats, genre and language table there are 2 primary keys and no other attributes depend on only part of it. Hence 2NF is also satisfied.

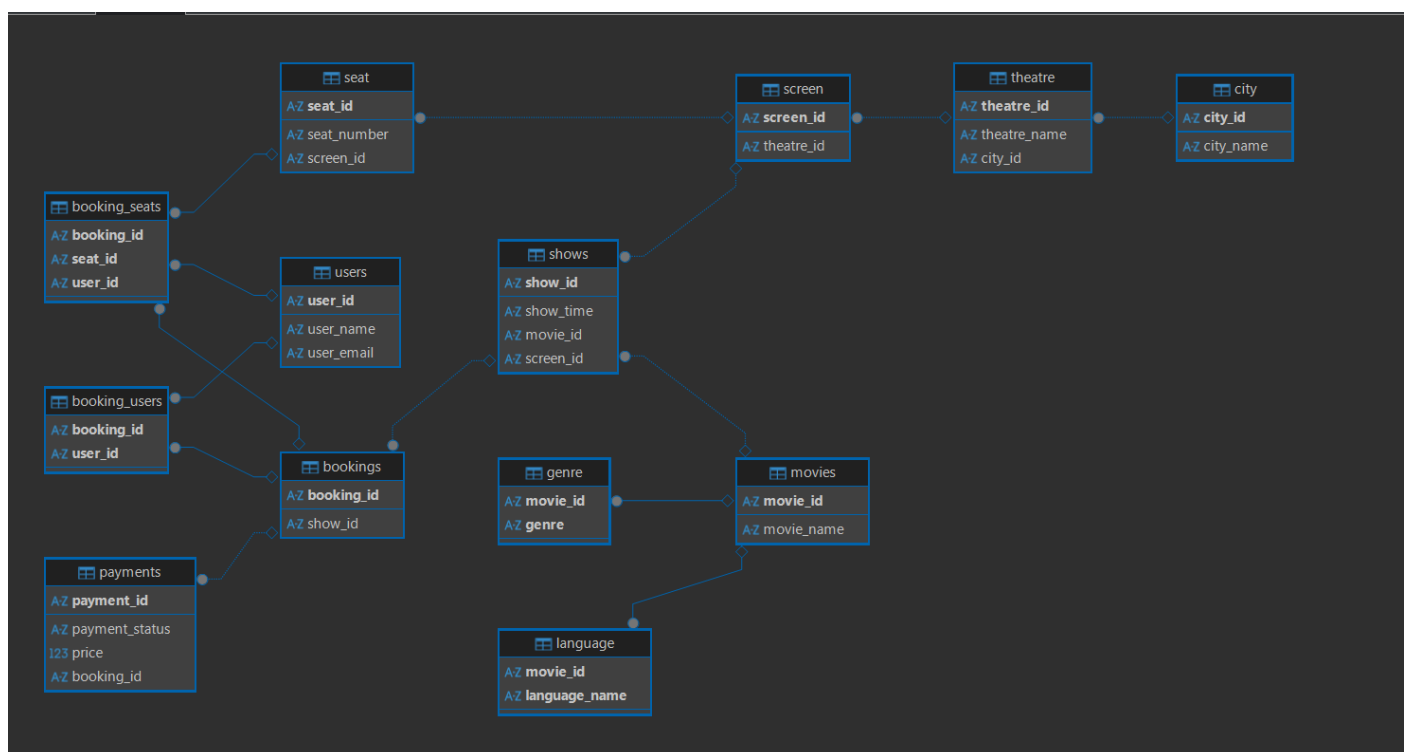
3NF states that there should not be transient dependency and this is satisfied in date scheme.

Update During Development

According to the changes updated the changes required in tables are

- Booking_users – booking_id and user_id [Here both are primary key and foreign key].
- Booking_seats – booking_id, seat_id and user_id [Here all of the attributes are primary and foreign keys]
- Booking – booking_id[PK] and show_id[FK]

ER Diagram



SECTION C

Queries and Transactions

1. Retrieve all bookings for a given user within a specified date range, including movie name, theatre, show time, and seats booked.

(Must use joins across relevant tables)

```
select
    u.user_name,
    m.movie_name,
    t.theatre_name,
    s.show_time,
    group_concat(se.seat_number) as seats_booked
from users u
join booking_users bu
    on u.user_id = bu.user_id
join bookings b
    on bu.booking_id = b.booking_id
join shows s
    on b.show_id = s.show_id
join movies m
    on s.movie_id = m.movie_id
join screen sc
    on s.screen_id = sc.screen_id
join theatre t
    on sc.theatre_id = t.theatre_id
join booking_seats bs
    on b.booking_id = bs.booking_id
    and bu.user_id = bs.user_id
join seat se
    on bs.seat_id = se.seat_id
where u.user_id = 'U1'
and s.show_time between '2024-01-01' and '2024-12-31'
group by u.user_name, m.movie_name, t.theatre_name, s.show_time, b.booking_id;
```

2. Identify the most frequently booked movie along with total bookings count.
(Must use aggregation and joins)

```
select
    m.movie_name,
    count(b.booking_id) as total_bookings
from bookings b
join shows s
    on b.show_id = s.show_id
join movies m
    on s.movie_id = m.movie_id
group by m.movie_name
order by total_bookings desc
limit 1;
```

3. For a given theatre and date, list all shows along with total seats booked and available seats.
(Must use joins and grouping)

```
select
s.show_id,
m.movie_name,
s.show_time,
count(distinct bs.seat_id) as seats_booked,
count(distinct se.seat_id) - count(distinct bs.seat_id) as seats_available
from shows s
join movies m
on s.movie_id = m.movie_id
join screen sc
on s.screen_id = sc.screen_id
join theatre t
on sc.theatre_id = t.theatre_id
join seat se
on sc.screen_id = se.screen_id
left join bookings b
on s.show_id = b.show_id
left join booking_seats bs
on b.booking_id = bs.booking_id
where t.theatre_id = 't1'
and date(s.show_time) = '2026-05-01'
group by s.show_id, m.movie_name, s.show_time;
```

4. Write a transaction to handle a booking operation:
- Check seat availability for a show
 - Insert booking record
 - Insert associated seat records
 - Update seat availability and Ensure the operation fails safely if seats are not available

```
start transaction;
select count(*) into @available
from seat se
where se.screen_id = 's1'
and se.seat_id not in (
    select bs.seat_id
    from booking_seats bs
    join bookings b on bs.booking_id = b.booking_id
    where b.show_id = 'sh1'
);

if @available < 2 then
    rollback;
else
    insert into bookings values ('b3', 'sh1');
    insert into booking_users values ('b3', 'u1');
    insert into booking_seats values ('b3', 'seat1', 'u1');
    insert into booking_seats values ('b3', 'seat3', 'u1');
    insert into payments values ('p3', 'paid', 500, 'b3');
```

```
        commit;  
end if;
```